

Práctica: Feature Stores

Objetivo

Esta práctica tiene como objetivo principal implementar el uso de feature stores para modelos que requieren *features* precomputados en el momento de inferencia.

Pre-requisitos

- Python

Materiales y Herramientas

- Editor de texto/IDE (VS Code, PyCharm, etc.).
 - Terminal/Línea de comandos.
-

1. Definiciones de Feast (**Feature Store** open-source)

Para esta práctica se propone incluir como *features* algunas agregaciones de las 10 lecturas más recientes de un pozo en particular.

Habiendo definido algunas como `std_prod_gas_10m` y `avg_prod_gas_10m` aparece una nueva incógnita: ¿cómo puedo tener precomputados tales features en el momento de realizar la inferencia?

Para esto se propone contar con un **offline store** en donde se almacenen los features históricos para realizar el entrenamiento, y además un **online store** donde sólo almacenaremos los features más recientes de cada pozo.

Para tal fin usaremos [Feast](#), un feature store open-source.

Dentro de nuestro directorio crearemos una carpeta con el siguiente contenido+

```
feature_store/  
├── data/  
└── feature_store.yaml
```

└─ features.py

Dentro de feature_store.yaml definiremos lo siguiente:

None

```
project: oil_gas_production
registry:
  /opt/airflow/feature_store/registry/registry.db
provider: local
offline_store:
  type: file
online_store:
  type: sqlite
  path:
    /opt/airflow/feature_store/online_store/online.db
```

Esto define el offline store como un archivo local, y el online store como una base de datos SQLite.

Luego, en features.py definiremos lo siguiente:

Python

```
from datetime import timedelta
from feast import Entity, FeatureView, Field,
FileSource
from feast.types import Float32, Int32

# — Entidad —

pozo = Entity(
    name="idpozo",
    description="Identificador único del pozo de
extracción",
)
```

```

# — Fuente de datos (offline store) —

well_stats_source = FileSource(

path="/opt/airflow/feature_store/data/well_features.pa
rquet",
    timestamp_field="fecha",
)

# — Feature View —

well_stats = FeatureView(
    name="well_stats",
    entities=[pozo],
    schema=[
        # - Features del dataset original -
        Field(name="prod_gas",      dtype=Float32), #
Target
        Field(name="prod_pet",      dtype=Float32), #
Target
        Field(name="prod_agua",     dtype=Float32),
        Field(name="tef",           dtype=Float32),
        Field(name="profundidad",   dtype=Float32),
        Field(name="tipoextraccion", dtype=Int32),

        # - Features de ventana (últimas 10 lecturas por
pozo) -
        Field(name="avg_prod_gas_10m", dtype=Float32),
        Field(name="avg_prod_pet_10m", dtype=Float32),
        Field(name="last_prod_gas",  dtype=Float32),
        Field(name="last_prod_pet",  dtype=Float32),
        Field(name="n_readings",     dtype=Int32),
    ],
    source=well_stats_source,
)

```

2. Populando los stores

Armamos un script en Python que descargue el dataset, lo guarde en un CSV, precompute los features históricos y los guarde en

`./feature_store/data/well_features.parquet`. Luego de eso ejecutar **feast apply** (comando que utiliza las definiciones de Feast del punto anterior y crea la db de metadata para luego poder usar el store) y luego popular los features más recientes de cada pozo en el online store.

Para esto se proporciona el siguiente snippet:

Python

```
import urllib.request
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder

DATA_DIR = "./feature_store/data"
CSV_PATH = DATA_DIR + "/dataset.csv"
PARQUET_PATH = DATA_DIR + "/well_features.parquet"

DATASET_DOWNLOAD_URL =
"http://datos.energia.gob.ar/dataset/c846e79c-026c-4040-897f-1ad3543b407c/resource/b5b58cdc-9e07-41f9-b392-fb9ec68b0725/download/produccion-de-pozos-de-gas-y-petrleo-no-convencional.csv"

def download_data():
    print("Descargando dataset...")
    urllib.request.urlretrieve(DATASET_DOWNLOAD_URL,
    CSV_PATH)
    print("Descarga completada.")

def prepare_offline_store():
```

```

    print("Preparando features históricas para el offline
store...")
    df = pd.read_csv(CSV_PATH)

    df['fecha'] = pd.to_datetime(
        df['anio'].astype(str) + '-' +
df['mes'].astype(str).str.zfill(2) + '-01'
    )

    columns = ['idpozo', 'fecha', 'prod_pet', 'prod_gas',
'tipoextraccion']
    df = df[columns].dropna()

    le = LabelEncoder()
    df['tipoextraccion'] =
le.fit_transform(df['tipoextraccion'])

    df = df.sort_values(['idpozo',
'fecha']).reset_index(drop=True)

    records = []

    for well_id, group in df.groupby('idpozo', sort=False):
        group =
group.sort_values('fecha').reset_index(drop=True)

        # Calculo los features agregando los 10 readings mas
recientes por pozo...

        # Creo una fila que represente la próxima ingesta.
        tail_window = group.iloc[max(0, len(group) - 10):
len(group)]
        if not tail_window.empty:

```

```

        online_rec = group.iloc[-1].to_dict()
        online_rec['fecha'] = online_rec['fecha'] +
pd.DateOffset(months=1)

        online_rec['prod_gas'] = None
        online_rec['prod_pet'] = None
        online_rec['avg_prod_gas_10m'] =
float(tail_window['prod_gas'].mean())
        online_rec['avg_prod_pet_10m'] =
float(tail_window['prod_pet'].mean())
        online_rec['last_prod_gas'] =
float(tail_window['prod_gas'].iloc[-1])
        online_rec['last_prod_pet'] =
float(tail_window['prod_pet'].iloc[-1])
        online_rec['n_readings'] =
int(len(tail_window))

        records.append(online_rec)

    feat_df = pd.DataFrame(records)
    feat_df.to_parquet(PARQUET_PATH, index=False)
    print(f"Features offline guardadas en {PARQUET_PATH} con
{len(feat_df)} filas.")

if __name__ == "__main__":
    download_data()
    prepare_offline_store()

```

Luego materializaremos el online store

Python

```
import os
import subprocess
import pandas as pd
from feast import FeatureStore

FEATURE_STORE_REPO = "./feature_store"
PARQUET_PATH = FEATURE_STORE_REPO +
"/data/well_features.parquet"

def populate_online_store():
    print("Materializando features al Online Store...")

    feat_df = pd.read_parquet(PARQUET_PATH)
    latest_df =
feat_df.sort_values('fecha').groupby('idpozo').tail(1)

    store = FeatureStore(repo_path=FEATURE_STORE_REPO)
    store.write_to_online_store(
        feature_view_name="well_stats",
        df=latest_df,
    )
    print("Materialización exitosa.")

# Demostración
print("\n[Online Store] Validando lectura...")
pozo_ejemplo = 132879

features = store.get_online_features(
    features=[
        "well_stats:avg_prod_gas_10m",
        "well_stats:n_readings"
    ],
```

```

    entity_rows=[{"idpozo": pozo_ejemplo}]
    ).to_dict()

    print(f"Lectura exitosa. Features del pozo
    {pozo_ejemplo}:")
    for key, value in features.items():
        print(f"  {key}: {value[0]}")

if __name__ == "__main__":
    populate_online_store()

```

3. Entrenando con un feature store

Utilicen el offline store para generar un modelo que utilice los features de los 10 readings más recientes:

Python

```

def train():
    print("Iniciando entrenamiento del modelo...")

    store = FeatureStore(repo_path=FEATURE_STORE_REPO)

    print("Leyendo llaves de entidades desde el
    parquet...")
    raw_df = pd.read_parquet(PARQUET_PATH)
    entity_df = raw_df[['idpozo', 'fecha',
    'prod_gas']].copy()

```



```

entity_df['fecha'] =
pd.to_datetime(entity_df['fecha'])
entity_df = entity_df.rename(columns={'fecha':
'event_timestamp'})

features = [
    'tipoextraccion', 'avg_prod_gas_10m',
    'avg_prod_pet_10m',
    'last_prod_gas', 'last_prod_pet', 'n_readings'
]
target = 'prod_gas'
feast_features = [f"well_stats:{f}" for f in
features]

print("Obteniendo features históricas desde el
Feature Store...")
training_df = store.get_historical_features(
    entity_df=entity_df,
    features=feast_features,
).to_df()

# Feast agrega prefijos "well_stats_...", los
removemos para facilitar
training_df.columns = [c.split('__')[-1] for c in
training_df.columns]

# Eliminamos nulos del target (puede haber nulos
por las filas "futuras" que agregamos en el offline
store para online request)
training_df = training_df.dropna(subset=[target])

X = training_df[features]
y = training_df[target]

```

```
# ...
```

4. Predicción con Feature Store

Al momento de predecir la producción de un pozo necesitamos obtener ciertos features. En este caso necesitamos los mas recientes. Convenientemente ya los materializamos en el online store.

Utilicen el online store para obtener los features más recientes de un pozo.

Python

```
def predict():

    store = FeatureStore(repo_path=FEATURE_STORE_REPO)
    idpozo_ejemplo = 132879

    print(f"Consultando contexto online para el pozo
{idpozo_ejemplo}...")
    online_features = store.get_online_features(
        features=[
            "well_stats:tipoextraccion",
            "well_stats:avg_prod_gas_10m",
            "well_stats:avg_prod_pet_10m",
            "well_stats:last_prod_gas",
            "well_stats:last_prod_pet",
            "well_stats:n_readings"
        ],
        entity_rows=[{"idpozo": idpozo_ejemplo}]
```

```
).to_df()
```

```
X_df = online_features[[  
    'tipoextraccion', 'avg_prod_gas_10m',  
    'avg_prod_pet_10m',  
    'last_prod_gas', 'last_prod_pet', 'n_readings'  
]]
```

```
# ...
```